

AIWear UI自动化测试报告

测试人员：本人

测试时间：2026年5月23日

测试项目：AIWear 系统

测试类型：UI 自动化测试

测试方式：Python + pytest + Selenium + YAML + Allure

一、测试概述

本报告基于已完成的 `AIWear_ui_auto_test` 测试工程、YAML 测试数据和 Allure 执行结果编写，主要记录 AIWear Web 端 UI 自动化测试的执行情况和测试结论。报告仅覆盖页面元素、页面跳转、按钮状态、输入校验和核心浏览器操作流程，不展开手工功能测试、接口自动化测试或性能测试内容。

二、测试目标

本次 UI 自动化测试目标如下：

- 验证 AIWear Web 端核心页面流程是否可用，包括登录、我的图片、文搜图、图搜图、图片编辑、图片合并和历史记录等页面。
- 验证主要 UI 交互是否符合预期，包括页面跳转、菜单高亮、标签切换、按钮状态、输入校验、结果展示和空态提示等。
- 通过自动化方式沉淀可重复执行的浏览器端回归测试资产，为后续页面变更和回归验证提供依据。

三、测试范围

本次 UI 自动化测试覆盖以下页面模块：

序号	测试模块	测试内容	用例数量
1	登录页面	验证码登录、密码登录、异常登录、登录态保持、退出登录、受保护页面跳转	9
2	我的图片页面	图片上传、图片卡片展示、标签切换、页面布局、菜单高亮、用户下拉菜单	9
3	文搜图页面	查询输入、搜索按钮状态、命中结果、空态文案、标签切换、菜单状态	9
4	图搜图页面	图片选择、搜索按钮状态、命中结果、空态文案、标签切换、菜单状态	9
5	图片编辑页面	图片选择、编辑指令输入、提交按钮状态、编辑结果展示、菜单跳转	10
6	图片合并页面	两张图片选择、合并指令输入、提交按钮状态、合并结果展示、菜单跳转	10
7	历史记录页面	表格展示、表头校验、记录内容展示、类型筛选、空占位、菜单状态	9

本报告不覆盖以下内容：

- 1. Web 页面手工功能测试结论。
- 2. 后端接口返回结构、接口字段、接口异常参数和接口鉴权逻辑的专项校验。
- 3. 接口并发性能、TPS、响应时间、吞吐量和稳定性压测结论。
- 4. 项目整体上线验收结论。

四、测试环境

本次 UI 自动化测试在已部署的 AIWear 测试环境中执行，前端页面、Java 后端服务、Python AI 服务和中间件均使用当前服务器部署环境。

配置项	配置值
被测系统地址	http://139.199.222.234
前端服务	通过 Nginx 对外提供 Web 页面访问
Java 服务	Spring Boot 后端服务，提供登录、图片管理、搜索、编辑、合并和历史记录等业务接口
Python 服务	Python AI 服务，提供图片搜索、图片编辑和图片合并等 AI 处理能力
中间件	MySQL、Redis、阿里云 OSS
Redis 用途	读取验证码等登录相关前置数据
浏览器	chrome
执行模式	headless: true
窗口大小	1600,1000
隐式等待	implicit_wait: 2
显式等待超时	timeout: 10
轮询间隔	poll_frequency: 0.5

测试图片素材包括人物图、服装图和动物图等，主要用于图片上传、图搜图、图片编辑和图片合并场景。

五、测试工具与技术

测试工程采用 `Python + pytest + Selenium + YAML + Allure` 的方式组织 UI 自动化测试。

工具或技术	用途
Python	编写 UI 自动化测试脚本和公共工具
pytest	测试用例收集、参数化执行、fixture 管理和断言执行
Selenium	驱动 Chrome 浏览器执行页面操作
PyYAML	读取环境配置、账号配置和 YAML 测试数据
allure-pytest	生成 Allure 原始结果和可视化测试报告
redis	读取验证码等登录相关前置数据

测试工程采用 Page Object 模式封装页面操作，将页面元素定位、点击、输入、等待和业务动作集中放在 `pages/` 目录中。测试数据采用 YAML 文件维护，测试脚本只负责读取数据、调用页面对象、执行断言和输出 Allure 结果。

六、测试用例设计

6.1 主要设计

本次 UI 自动化测试共执行 65 条用例，用例按页面模块拆分，每个模块使用独立的 YAML 文件维护测试数据，并通过 pytest 参数化方式执行。

6.2 用例设计重点

1. 登录与会话：覆盖验证码登录、密码登录、异常登录、登录态保持和退出登录。
2. 图片业务流程：覆盖图片上传、文搜图、图搜图、图片编辑和图片合并等核心操作。
3. 页面交互状态：覆盖按钮可用状态、占位文案、空态提示、标签切换、菜单高亮和页面跳转。
4. 历史记录展示：覆盖记录表格、记录内容、类型筛选和结果图展示。

6.3 用例数据来源

测试用例数据来自 `AIWear_ui_auto_test/data` 目录：

数据文件	覆盖模块	用例数量
login.yaml	登录页面	9
my_images.yaml	我的图片页面	9
text_search.yaml	文搜图页面	9
image_search.yaml	图搜图页面	9
edit.yaml	图片编辑页面	10
merge.yaml	图片合并页面	10
records.yaml	历史记录页面	9

测试脚本位于 `AIWear_ui_auto_test/tests` 目录，主要包括：

代码块

```
1  tests/
2  |— test_login.py
3  |— test_my_images.py
4  |— test_text_search.py
5  |— test_image_search.py
6  |— test_edit.py
7  |— test_merge.py
8  |— test_records.py
```

七、测试脚本设计与实现

UI 自动化工程主要由执行配置、公共 fixture、页面对象、测试数据、测试用例、公共工具和报告目录组成。

简化后的项目框架如下：

代码块

```
1  AIWear_ui_auto_test/
2  |— pytest.ini          # pytest 收集规则和 Allure 输出配置
3  |— conftest.py         # 全局 fixture、浏览器、登录态、测试图片准备、失败截图
4  |— README.md          # UI 自动化工程说明
5  |— requirements.txt    # Python 依赖清单
6  |— config/            # 环境、账号、日志配置
7  |— data/              # YAML 测试数据
8  |— images/            # UI 自动化测试图片素材
9  |— pages/            # Page Object 页面操作封装
10 |— tests/             # pytest UI 自动化测试用例
11 |— utils/            # driver、YAML、Redis、截图、日志等公共工具
12 |— report/           # Allure 原始结果和静态报告
13 |— logs/             # UI 自动化执行日志
```

关键实现说明如下：

1. `pytest.ini` 定义测试收集目录、测试文件命名规则、测试类和测试函数命名规则，并配置 Allure 原始结果输出目录。
2. `conftest.py` 负责加载环境配置、账号配置和测试图片路径，创建浏览器驱动，复用登录态，准备图片素材，并在测试失败时自动截图后挂载到 Allure 报告。
3. `pages/` 采用 Page Object 模式封装页面操作，使测试脚本重点关注业务流程和断言，不直接堆叠 Selenium 细节。
4. `tests/` 按页面模块组织测试用例，通过 `pytest.mark.parametrize` 读取 YAML 数据后执行对应页面操作和断言。
5. `data/` 按模块维护测试数据，使输入参数、预期结果和标记信息从 Python 脚本中拆分出来，便于后续维护。
6. `utils/` 提供浏览器创建、YAML 读取、Redis 读取、日志记录、显式等待和失败截图等公共能力。
7. `report/` 保存 Allure 原始结果和静态报告，用于查看用例执行状态、耗时、参数和失败信息。

脚本执行流程如下：

代码块

```
1  pytest
2  -> 读取 pytest.ini
3  -> 收集 tests/test_*.py
4  -> 加载 conftest.py 中的 fixture
5  -> 初始化 driver / logged_in_driver / prepared_images
```

- 6 -> 测试函数读取 data/*.yaml 中的场景数据
- 7 -> 调用 pages/ 中的页面对象执行浏览器操作
- 8 -> 在 tests/ 中执行页面状态和业务结果断言
- 9 -> 将执行结果写入 Allure 原始结果目录
- 10 -> 生成 Allure 可视化报告

八、测试执行情况

本次报告依据已完成的 UI 自动化执行结果编写，未重新执行测试。

8.1 测试执行配置

代码块

```
1 addopts = -vs --alluredir=report/allure-results --clean-alluredir
2 testpaths = tests
3 python_files = test_*.py
4 python_classes = Test*
5 python_functions = test_*
```

8.2 Allure 报告生成命令

代码块

```
1 pytest
2 allure generate report/allure-results -o report/allure-report --clean
```

8.3 执行结果汇总

指标	结果
总用例数	65
通过	65
失败	0
阻塞	0
跳过	0
未知	0
通过率	100%
执行开始时间	2026/5/23 11:08
执行结束时间	2026/5/23 11:11
总耗时	约 170 秒

8.4 分模块执行结果

测试模块	用例数量	通过	失败	通过率
登录页面	9	9	0	100%
我的图片页面	9	9	0	100%
文搜图页面	9	9	0	100%
图搜图页面	9	9	0	100%
图片编辑页面	10	10	0	100%
图片合并页面	10	10	0	100%
历史记录页面	9	9	0	100%

九、测试结果分析

从执行结果看，本次 UI 自动化测试覆盖的 65 条用例全部通过，说明当前测试环境下 AIWear Web 端核心页面流程和主要交互行为符合预期。

- 1. 登录、页面跳转、菜单高亮、标签切换等基础交互表现正常。
- 2. 图片上传、搜索、编辑、合并和历史记录等核心业务流程可以正常完成。
- 3. 按钮状态、空态提示、结果展示和记录筛选等页面反馈符合当前测试预期。

本次结果仅代表 UI 自动化覆盖范围内的页面验证结论，不等同于接口自动化或性能测试结论。

十、缺陷记录

本次 UI 自动化执行结果中未出现失败、阻塞或跳过用例，未记录由本次自动化执行直接发现的 UI 缺陷。

未发现缺陷只代表本次 UI 自动化用例覆盖范围内未出现失败结果，不代表系统不存在所有潜在 UI 问题。

十一、测试结论

本次 AIWear UI 自动化测试共执行 65 条用例，65 条全部通过，通过率为 100%。测试覆盖登录、我的图片、文搜图、图搜图、图片编辑、图片合并和历史记录等核心 Web 页面模块。

根据本次执行结果，可以得出以下结论：

- 1. 当前测试环境下，UI 自动化覆盖范围内的核心页面流程可以正常完成。
- 2. 页面元素展示、菜单高亮、标签切换、按钮状态、空态提示和结果展示符合当前测试预期。
- 3. 登录态保持、退出登录、受保护页面跳转等关键页面行为已通过自动化用例验证。
- 4. 图片上传、图片搜索、图片编辑、图片合并和历史记录筛选等主要浏览器端业务流程已通过自动化用例验证。
- 5. UI 自动化测试结果可作为 AIWear 项目测试报告中 **UI自动化测试** 专项测试依据。

综合判断：AIWear 系统 Web 端在本次 UI 自动化测试覆盖范围内满足测试目标。

十二、附件说明

12.1 UI 自动化源码:

<https://github.com/23YuTingYang/AIWear-ui-auto-test>

12.2 Allure 测试报告:

<https://23yutingyang.github.io/aiwear-ui-allure-report/>